

Learning Objectives

- Understand how a stack can be implemented using an array and how `push` and `pop` manage its contents.
- Understand that variables and array elements occupy locations in memory and that these locations have addresses.
- Understand how pointers can be used to refer to and access values stored in memory.
- Understand pointer arithmetic and explore its relationship to arrays and array indexing.

Step 1 — Building a Stack with an Array

In the lecture videos, you learned how arrays can be used to store several values of the same type. We will now use an array to implement a simple stack that can store up to five integer values.

A stack provides two basic operations:

- `push` adds a value to the top of the stack.
- `pop` removes the value at the top of the stack and returns it.

The variable `stackSize` keeps track of how many elements are currently stored on the stack. Study the implementation in `stack.abc` below before continuing. Try to understand what happens to `stack` and `stackSize` when `push()` or `pop()` is called.

```
@ <stdio.h>

global
    stack: array[5] of int,
    stackSize: int = 0;

fn push(val: int)
{
    assert(stackSize < sizeof(stack) / sizeof(stack[0]));
    stack[stackSize++] = val;
}

fn pop(): int
{
    assert(stackSize > 0);
    return stack[--stackSize];
}

fn main()
{
    push(3);
    push(4);
    pop();
}
```

Step 2 — Accessing the Top of the Stack

So far, `pop()` is our only operation for accessing the value at the top of the stack. However, `pop()` also removes this value from the stack. Implement a function

```
fn top(): int
```

that returns the value at the top of the stack **without removing it**. As with `pop()`, calling `top()` on an empty stack is not allowed. Use `assert()` to detect this case.

Test your implementation with the following main function:

```
fn main()
{
    push(3);
    printf("top = %d\n", top());
    push(4);
    printf("top = %d\n", top());
    pop();
    printf("top = %d\n", top());
}
```

This program should produce the following output:

```
lehn$ ./a.out
top = 3
top = 4
top = 3
```

Compare your implementations of `top()` and `pop()`. What is the essential difference between the two operations? In particular, what happens to `stackSize`?

Step 3 — Inspecting the Stack

So far, we have only looked at the value at the top of the stack. To inspect all values that are currently stored on the stack, add the following function:

```
fn info()
{
    printf("stack: ");
    for (local i: int = 0; i < stackSize; ++i) {
        printf("[%d] ", stack[i]);
    }
    printf("\n");
}
```

Modify `main` so that after each `push` or `pop` operation, both the value at the top of the stack and the complete stack contents are displayed. For example, the operations

```
push(3);
push(4);
pop();
```

should produce output similar to:

```
lehn$ ./a.out
top = 3
stack: [3]
top = 4
stack: [3] [4]
top = 3
stack: [3]
```

Step 4 — What Does `pop()` Actually Do?

After `push(3)`, `push(4)`, and `pop()`, the `info()` function displays

```
stack: [3]
```

But what actually happened to the value 4?

Remember that our stack is implemented using an array with five elements. The current implementation of `info()` only displays those array elements that are currently part of the stack.

Extend `info()` so that it also displays the remaining elements of the underlying array. Separate the elements that currently belong to the stack from the remaining elements of the underlying array by a vertical bar `|`. Your implementation of `info()` should not depend on the array having exactly five elements. If the size of `stack` is changed, for example from 5 to 10, `info()` should continue to work without any other changes.

Your implementation of `info()` should not depend on the array having exactly five elements. If the size of `stack` is changed, for example from 5 to 10, `info()` should continue to work without any other changes.

```
MCL:tmp lehn$ ./a.out
top = 3
stack: [3] | [0] [0] [0] [0]
top = 4
stack: [3] [4] | [0] [0] [0]
top = 3
stack: [3] | [4] [0] [0] [0]
```

Look carefully at the last line. What happened to the value 4 when `pop()` was called? Compare this observation with your answer about `stackSize` in Step 2. What will happen to the value 4 when another value is pushed onto the stack?

Step 5 — Using Our Stack for Function Calls

In Worksheet 3, we saw that each function call gets its own stack frame containing the data associated with that call. In particular, recursive calls require several stack frames to exist at the same time.

We will now use the simple stack implemented in the previous steps to simulate one small part of this behavior: storing the function parameter `n`.

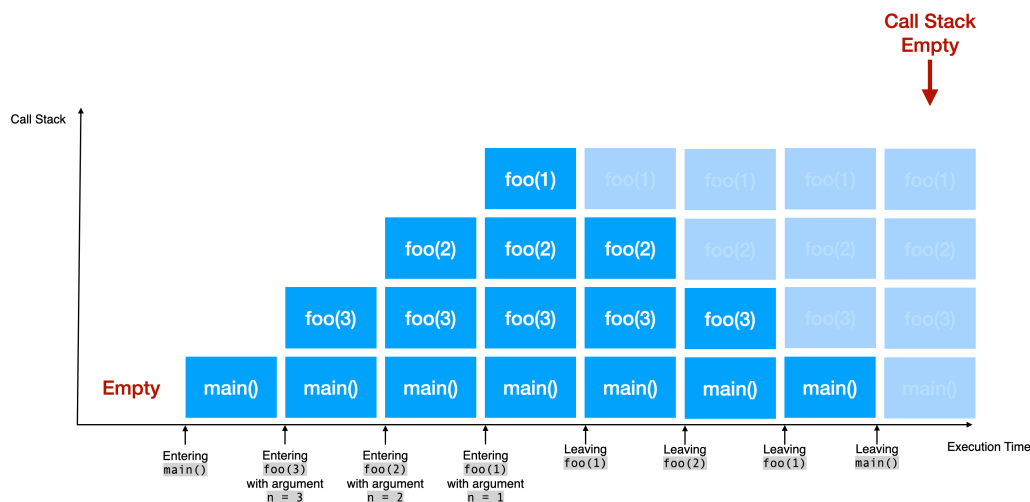
Consider the following recursive program:

```
@ <stdio.h>

fn foo(n: int)
{
    printf("entered foo with n = %d\n", n);
    if (n > 1) {
        foo(n - 1);
    }
    printf("leaving foo with n = %d\n", n);
}

fn main()
{
    foo(3);
}
```

The following figure from Worksheet 3 illustrates the stack frames that exist during the execution of this program.



Task

We will now simulate this behavior using the stack that we implemented in the previous steps. Modify the program according to the following rules:

- Replace a call such as `foo(3)` by

```
push(3);
foo();
```

The argument is therefore placed on the stack before the function is called.

- Change `foo` so that it no longer has a parameter. Whenever the function needs the value of `n`, obtain it from the top of the stack using `top()`.
- Before `foo()` returns, remove the value belonging to this call from our stack using `pop()`. Remember from Step 4 that `pop()` does not erase the value from the underlying array—it only removes it from the active part of the stack.
- Use `info()` when entering and leaving `foo()` to observe how the stack changes during the recursive calls. Pay particular attention to the array elements that are no longer part of the stack after a call returns.

After all calls have returned, our stack is empty. Are the values `3`, `2`, and `1` still present in the underlying array? Compare this with the faded stack frames in the figure above. What does “removing” a value from the stack actually mean?

Step 6 — Pointers and Pointers to Pointers

In the lecture videos, you saw that every variable occupies a location in memory and that each memory location has an address. A pointer is a variable that stores such an address.

Consider the following program (ex1.abc). Before running it, try to understand how the variables `a`, `b`, `p`, and `q` are related.

```

1 @ <stdio.h>
2
3 fn main()
4 {
5     local
6     a: int = 42,
7     b: int = 123,
8     p: -> int = &a,
9     q: -> -> int = &p;
10
11     printf("a = %d\n", a);
12     printf("b = %d\n", b);
13     printf("*p = %d\n", *p);
14     printf("***q = %d\n\n", **q);
15
16     // here some magic will happen ;-)
17
18     *p = 13;
19     printf("a = %d\n", a);
20     printf("b = %d\n\n", b);
21
22     **q = 31;
23     printf("a = %d\n", a);
24     printf("b = %d\n", b);
25 }
```

Task

1. Draw the memory layout of the variables `a`, `b`, `p`, and `q` when execution reaches line 10. For each variable, indicate the value stored in its memory location. For pointers, use arrows to show which memory location they refer to.
Then predict how this memory layout changes when execution reaches lines 21 and 25.
2. Replace the comment in line 16

```
// here some magic will happen ;-)
```

with a single statement such that the assignments in lines 18 and 22 modify `b` instead of `a`.
Before running the program, explain why your statement should have this effect. Then test your prediction.
3. Add `printf` statements after line 14 to investigate the actual memory layout. Use appropriate format specifiers to print
 - A. the sizes of `a`, `b`, `p`, and `q`,
 - B. the addresses of `a`, `b`, and `p`, and
 - C. the values stored in `p` and `q`.
4. Compare the output from Task 3 with the memory layout you drew in Task 1. In particular, identify which addresses are stored in `p` and `q`, and explain what the expressions `*p`, `*q`, `**q` refer to.